SPECDEC CORRECTNESS FORENSICS

从一次 Token 分歧到 CUDA BF16 形状敏感性

SpecDec 正确性问题的发现、归因、候选评估与最终处理

报告目的

这份报告不把调试过程写成按日期排列的日志，而是重建一条可验证的推理链：每个实验为什么被提出、固定了哪些变量、测到了什么、排除了什么，以及为什么结果自然导向下一项实验。

项目：Qwen2.5 目标模型上的投机解码与 KV 缓存研究
平台：NVIDIA GeForce RTX 4060 Laptop GPU 8GB
定位配置：BF16、all-eager、ctx=8192、k=4；当前候选：仅 prefill cuDNN 融合
更新日期：2026-10-07；保留 10 月 3 日定位过程，新增后续收尾证据



最终归因链：数学语义一致，不代表有限精度下跨形状逐位一致

目录

目录 2

1 执行摘要 4

1.1 最终归因链 . . . 4

1.2 一句话回答四个问题 . . . 4

2 背景：为什么一次 token 不同就是正确性问题 4

2.1 本报告中的两个执行形状 . . . 5

2.2 如何阅读关键指标 . . . 5

3 归因方法：怎样避免把相关性误写成原因 5

3.1 五条实验纪律 . . . 5

3.2 三类最关键的实验设计 . . . 5

4 定位时间线：从异常输出到可重复现场 6

4.1 最初症状：AR 与 Spec 生成不一致 . . . 6

4.2 先修掉一个真实问题：RoPE inv_freq 来源 . . . 6

4.3 历史效应：位置 8239 的 cache 已经不同 . . . 6

4.4 串行重建把新首分歧提前到位置 8209 . . . 7

5 同历史实验：把 cache 历史和执行形状真正拆开 7

5.1 位置 8209 的 2×2 . . . 7

6 从输出向输入上移：找到首次分歧层 7

6.1 LM head 边界 . . . 8

6.2 Final RMSNorm 边界 . . . 8

6.3 L27 四格 . . . 8

6.4 全层边界捕获 . . . 8

6.5 全层差异如何演化 . . . 9

7 进入 L0：从首个算子差异到持续性来源 9

7.1 L0 内部全链捕获 . . . 9

7.2 L0 关键节点 . . . 9

8 QK 2×2：把 query 行数与 key 长度拆开 9

9 因果干预：把相关性变成可验证的恢复链 10

9.1 serial-causal QK . . . 10

9.2 再对齐 o_proj . . . 10
9.3 闭合 MLP 的 gate/up/down . . . 11
9.4 最终 L0 闭环表 . . . 11

10 最终技术解释 11

10.1 这不是一个孤立 bug . . . 11
10.2 为什么不同形状会产生不同结果 . . . 11
10.3 为什么一个 ULP 能改变 token . . . 12

11 局部定位实验排除了什么 12

12 机制结论的边界与后续验证 12

13 从局部修复到候选验证：为什么没有直接解决全部分歧 13

13.1 全层 row-aligned 候选 . . . 13
13.2 四个当时未见 family 的泛化检查 . . . 13

14 最后采用的方法：serial oracle 与生产路径分开 14

14.1 一次 serial 验证在数据上改变了什么 . . . 14
14.2 两轮 oracle 证据不能混成一个无条件大 PASS . . . 14

15 B6 收尾：覆盖、验收器修正与真实 EOS 15

15.1 不是只有一张 token equality 表 . . . 15
15.2 两个 harness 问题：修规则，不改数值以迁就测试 . . . 15
15.3 EOS cache 长度为什么可以差 1 . . . 15

16 最终如何判定与报告，而不是怎样掩盖差异 16

16.1 截至本次更新的状态 . . . 16
16.2 有哪些说法必须修正 . . . 16

17 从正确性收尾到正式实验 17

17.1 复盘时按这个顺序追证据 . . . 17

附录 A 关键实验与提交时间线 18

附录 B 关键产物索引 18

附录 C 复盘时应记住的五句话 19

1 执行摘要

项目在 8K 上下文的投机解码测试中发现：target-only 自回归解码与 SpecDec 块验证有时会生成不同 token。最初这看起来像候选行索引、causal mask、KV cache 回滚或 cuDNN 接入错误。经过多轮受控实验，受控案例支持：对应结构检查通过，BF16 运算的执行形状敏感性可以独立产生并传播差异，但不能把这个案例扩写为全部生产分歧的唯一根因。AR 的 GEMM 通常处理一个 token，M=1；块验证同时处理四个 token，M=4。即使对应行输入和权重逐位相同，不同形状也可能采用不同 kernel、tile 或归约顺序，产生 1 至 2 ULP 的差异。

在位置 8209 的同历史实验中，单步路径选择 token 323，四行块路径选择 token 659。两者仅相差一个 BF16 步长，且单步路径中两个候选精确并列。通过从输出向输入逐步上移边界，团队把首次分歧定位到 L0，再进入 L0 内部，识别出 QK matmul、o_proj、gate_proj、up_proj 和 down_proj 五个持续性 GEMM 边界。逐项把这些操作改成每行 M=1 后，L0 输出最终与 AR 逐位一致；随后差异在 L1 重新出现。

当前状态

局部机制已建立因果证据；后续逐行候选未形成通用输出保证。项目最终采用 serial oracle 检查控制逻辑、生产批量路径记录数值行为的分层方法，BF16 B6 已获有限范围批准。生产 AR/Spec exact-id 差异没有被消除；pilot 和正式采集仍未完成。

1.1 最终归因链



最终归因链：数学语义一致，不代表有限精度下跨形状逐位一致

这里最重要的区分是：数学语义相同并不等于有限精度实现逐位相同。SpecDec 的块验证在数学上与逐 token 验证等价，但 GPU BF16 kernel 并不承诺跨矩阵形状的逐位不变性。近并列决策把本来很小的数值差异放大成了离散 token 差异。

1.2 一句话回答四个问题

问题	结论
是不是 SpecDec 接受/拒绝逻辑写错?	未发现。候选、判断行、replacement、rollback、cache 长度与结构不变量均通过。
是不是 KV cache 或 causal mask 错?	在关键同历史实验中排除。共同历史逐位相同，决策行可见范围正确，未来槽位无泄漏。
是不是 cuDNN 引入的问题?	不是唯一原因。问题在 all-eager 路径稳定复现。
最终找到的机制是什么?	CUDA BF16 GEMM 在 M=1 与 M=4 时产生相邻 ULP 差异，经过层间传播，在近并列 logits 上改变 argmax。

2 背景：为什么一次 token 不同就是正确性问题

投机解码先由较小 draft 模型提出一段候选 token，再由 target 模型一次性验证。若候选与 target 一致，可以一次提交多个 token，从而减少昂贵的 target 调用。理论上，贪心 SpecDec 应生成与 target-only greedy 相同的 token 序列。项目的 B6 门因此关注三件事：生成 token IDs 一致、所有数值有限、重复运行确定。

这并不要求所有 logits 逐位相同，但 token 不同意味着算法在当前有限精度实现下没有复现基线行为。尤其在后续性能实验中，一旦 AR 与 Spec 生成不同文本，接受率、轮数、缓存长度和运行时间都会改变，速度比较也会混入工作量差异。因此正确性门必须先于正式性能声明。

2.1 本报告中的两个执行形状

路径	典型输入	GEMM 的 M	语义
S / AR 单步	[1,1,H]	1	当前位置只计算一个 token
B / Spec verify	[1,4,H]	4	一次验证四个候选 token

```
Q = X @ Wq.T + bq
K = X @ Wk.T + bk
V = X @ Wv.T + bv

S path: [1,1536] @ [1536,1536]
B path: [4,1536] @ [1536,1536]
```

2.2 如何阅读关键指标

指标	含义	本项目中的用途
n_diff	逐位不相等的元素个数	定位差异首次出现和传播规模，不能当作质量距离。
max|Δ|	最大绝对差	判断是否为粗差，或仅为相邻 BF16 格点。
ULP	当前数值量级下相邻浮点数间距	说明 0.125、0.00390625 等差异有多小。
margin	top-1 与 top-2 logit 差	判断 token 决策是否贴边；margin=0 表示精确并列。
exact_equal	torch.equal 的逐位一致	建立强控制链，证明输入、缓存或中间边界相同。
argmax	最终选择 token	正式 greedy 行为；并列时取第一个最大值，通常是较小索引。

一个重要误区

n_diff 并不单调。对齐更多算子后，最终 logits 的 n_diff 有时反而增加，因为误差会在非线性层中重新组合、抵消或放大。真正的因果结论来自受控干预，不来自‘差异数量看起来更小’。

3 归因方法：怎样避免把相关性误写成原因

整个定位过程的核心不是增加日志，而是不断构造更强的反事实。每一轮都固定尽可能多的变量，只允许一个因素变化，并设置复现门和逐位控制。这样，结果才能回答‘如果只改变 X，Y 会不会变化’。

3.1 五条实验纪律

- 先复现现场，再做定位。若 token、位置、输入 hash 或历史产物不能复现，立即停止。
- 把历史内容与执行形状分开。不同 cache 历史不能用于判断 single/block 的直接效应。
- 边界从输出向输入上移。先问差异进入模块前是否已经存在，再决定是否进入模块内部。
- 先做观察，再做干预。观察能定位边界；只有把边界对齐并看下游是否恢复，才能建立因果闭环。
- 实测、支持性推断、未证实严格分栏。近并列或 ULP 解释不能自动豁免正式正确性门。

3.2 三类最关键的实验设计

设计	回答的问题	示例
同历史固定输入	排除 cache 历史内容，只比较执行形状	H(8208) 上 S=[20642] 与 B=[20642,659,576,4531]
2×2 因子分解	把同时变化的因素拆开，检查主效应与交互	QK 的 query 行数 M∈{1,4} 与 key 长度 N∈{8209,8212}
逐项因果干预	把某个边界恢复成 AR 几何，看下游是否恢复	serial QK、serial o_proj、serial MLP projections

从输出向输入逐步上移边界



每一步都先问：差异进入该模块前是否已经存在？固定输入后，执行形状能否单独翻转结果？

4 定位时间线：从异常输出到可重复现场

4.1 最初症状：AR 与 Spec 生成不一致

研究问题	投机解码为什么没有严格复现 target-only greedy?
为什么这样设计	在固定 F13、ctx=8192、k=4、max_new=64 的真实生成中，对 AR 与 Spec 使用相同模型权重、tokenizer、上下文和精度，比较完整 token 序列，而不是只看文本片段。
关键控制	两臂独立进程、空缓存开始、相同 prompt hash、相同停止规则、全 finite、结构不变量通过。
观测结果	RoPE 对齐后的真实生成共同前缀为 47，首次分歧在 index 47、绝对位置 8239：AR=6702，Spec=6652；64 token 中只差 1 个。
结果如何推动下一步	差异很小但稳定，说明需要定位一个易翻转决策，而不能用‘文本看起来差不多’放行。

4.2 先修掉一个真实问题：RoPE inv_freq 来源

研究问题	自研引擎与 Hugging Face 的 prefill 为什么不同?
为什么这样设计	逐层比较自研与 HF 的 KV，发现 L0 K 有 60 个差异，且只落在同一个 RoPE 旋转对。进一步固定输入、权重和日程，只改变 inv_freq 的计算来源：GPU-FP32 改为 CPU-FP32 后再搬到 GPU。
关键控制	权重 338/338 逐位一致；K/V shape、chunk 日程、BF16 与 reduction 开关一致；只替换 inv_freq 来源并在 finally 中恢复。
观测结果	对齐后 28 层 K/V 共 56 个张量全部与 HF 逐位一致，末位 151936 维 logits 也逐位一致。
结果如何推动下一步	这证明 RoPE 来源确实是一个真实缺陷。但真实 AR/Spec 仍在位置 8239 分歧，因此它不是剩余问题的完整解释。

4.3 历史效应：位置 8239 的 cache 已经不同

研究问题	分歧当轮是块验证直接算错，还是此前的数值差异已经写入 KV 历史?
为什么这样设计	在首个真实分歧轮捕获 AR 与 Spec 的完整 target cache，用同一种 single 或 block 执行分别搭配两份历史，形成‘历史 × 执行’四格。
关键控制	预测位置、判断行、可见 key、replacement、rollback、泄漏阳性对照和 cache 层数全部核对。
观测结果	固定执行方式、只换历史即可让 argmax 6702□6652 翻转；同一历史下 single/block 不改变该行 argmax。两份历史从第一个生成 token 起开始分叉。
结果如何推动下一步	这说明位置 8239 的直接决定因素是既有历史，但还没有回答最早的数值差异怎样产生。于是下一步必须构造更早、真正同历史的现场。

4.4 串行重建把新首分歧提前到位置 8209

研究问题	如果 Spec 每轮提交的 token 用 target 单步重放，新的历史会在哪里首次与原轨迹分开?
为什么这样设计	每轮在 verify 前深拷贝 H_pre，保留原接受/拒绝结果，再把实际 emitted token 逐个喂入 target，使用重放后的 cache 和 saved_next 进入下一轮。
关键控制	feeds 必须逐 token 等于 emitted；cache 长度必须为 pre+len(emitted)；draft 路径不改；重建与生产字段分开。
观测结果	重建轨迹与 AR/原 Spec 在 index 17、位置 8209 分开：重建为 659，AR 和原 Spec 为 323。新分歧前 [0,8208) 的 56 个 KV 张量逐位一致。
结果如何推动下一步	位置 8209 提供了一个更干净的同历史现场，可以直接把执行形状作为唯一变量。

5 同历史实验：把 cache 历史和执行形状真正拆开

5.1 位置 8209 的 2×2

固定同一份 H(8208)，当前位置 token 都是 20642。S 路径只喂一个 token；B 路径喂四个 token，但只比较 B 的第 0 行，因为它预测同一个位置 8209，且只能看到相同的历史前缀。另一个历史维度用串行历史与原 Spec 块历史构成四格。

历史	执行	argmax	margin	logit(323)	logit(659)
串行 H(8208)	S 单步	323	0.0	23.75	23.75
串行 H(8208)	B 四行	659	0.125	23.625	23.75
块保留 H(8208)	S 单步	323	0.0	23.75	23.75
块保留 H(8208)	B 四行	323	0.0	23.75	23.75

这一结果第一次证明：在完全相同的历史、相同 token、相同权重与相同可见范围下，单步和四行块执行可以直接改变 argmax。串行历史的 B 格把 token 323 的 logit 从 23.75 推到 23.625，恰好下降一个 BF16 ULP。

为什么微小差异会变成不同 token

并列时 torch.argmax 选择第一个最大值，因此 S 格选 323；B 格中 659 比 323 高 0.125，因此选 659。这里的 token 翻转不是大幅语义变化，而是一个离散决策落在浮点格点边界上。

6 从输出向输入上移：找到首次分歧层

从输出向输入逐步上移边界



每一步都先问：差异进入该模块前是否已经存在？固定输入后，执行形状能否单独翻转结果？

6.1 LM head 边界

研究问题	是隐藏状态不同，还是同一个隐藏状态用单行/四行词表投影导致 argmax 翻转?
为什么这样设计	固定 LM head 的输入来源和投影形状做两向替换：同输入换形状、同形状换输入。
关键控制	使用同一绑定 embedding 权重；完整 151936 维词表投影；真实组合必须复现 S/B。
观测结果	投影形状会改变少量 logits，但不会改变 token；替换隐藏向量来源会翻转 323/659。
结果如何推动下一步	LM head 不是首次来源，差异已经存在于最终隐藏向量中。边界继续上移到 Final RMSNorm。

6.2 Final RMSNorm 边界

研究问题	Final RMSNorm 是否因输入行数不同产生差异?
为什么这样设计	对相同隐藏向量分别以单行和四行调用同一 RMSNorm，再统一使用单行 LM head。
关键控制	RMSNorm 权重、eps、dtype 一致；比较对应第 0 行；真实输入/输出链逐位核对。
观测结果	固定输入时 RMSNorm 第 0 行逐位不受行数影响；输入来源不同则 token 翻转。
结果如何推动下一步	差异进入 Final RMSNorm 前已经存在，继续上移到最后一个 DecoderLayer L27。

6.3 L27 四格

研究问题	最后一层自己产生了翻转，还是接收了上游已经不同的隐藏状态?
为什么这样设计	构造输入来源 aS/aB 与执行形状 single/block 的四格，每格独立克隆 L27 轮前 cache，输出统一接同一 Final RMSNorm 和 LM head。
关键控制	embedding、cos/sin、L27 轮前 cache 逐位相同；真实 S/B 两格必须逐位复现。
观测结果	L27 输入已在 1357/1536 个维度不同，max|Δ|=0.5。固定输入换形状会改变数值，但不会翻转 token；固定形状换输入会翻转。
结果如何推动下一步	L27 不是首次来源。为了避免逐层猜测，下一步一次捕获全部 28 层边界。

6.4 全层边界捕获

研究问题	差异究竟在哪一层第一次从相同输入变成不同输出?
为什么这样设计	同一次真实 S/B 前向中，用只读 pre/post hook 保存每层决策行输入和输出；比较全部 28 层，而不是依赖误差单调假设做二分。
关键控制	L0 输入必须等于 embedding；每层输出必须逐位等于下一层输入；L27 输出必须等于 Final RMSNorm 输入；两次重复逐位一致。
观测结果	L0 输入逐位相同，L0 输出已有 111/1536 个差异，max|Δ|=0.00390625。L1 输入等于 L0 输出，之后差异扩大并波动。
结果如何推动下一步	首次分歧层确定为 L0。下一步只进入 L0 内部，不再扫描其他层。

6.5 全层差异如何演化

边界	n_diff	max|Δ|	解释
L0 input	0	0	embedding 决策行逐位相同
L0 output	111	0.00390625	第一层首次产生差异
L1 output	1234	0.03125	差异迅速扩散
L10 output	1365	0.25	规模扩大但不单调
L19 output	1318	0.75	max 差异继续变化
L27 output	1386	2.0	进入 Final RMSNorm 前已广泛不同
Final logits	136478	0.25	最终 argmax 323/659 不同

7 进入 L0：从首个算子差异到持续性来源

7.1 L0 内部全链捕获

研究问题	L0 内第一个输入相同、输出不同的算子是什么?
为什么这样设计	在真实生产路径中增加默认关闭的只读 probe，捕获 input RMSNorm、Q/K/V projection、RoPE、QK、mask、softmax、AV、o_proj、残差与 MLP 各边界。只比较语义对应的决策行或共同 key 前缀。
关键控制	L0 输入、cache、cos/sin、权重相同；probe 开关不改变输出；两次捕获逐位一致。
观测结果	q_proj 首先出现 2/1536 个差异，max|Δ|=0.000244140625；但 RoPE(Q) 后差异被吸收。随后 QK 在 Q/K 输入重新逐位一致时又产生 3 个 score 差异，max|Δ|=4。
结果如何推动下一步	这揭示了‘最早瞬时差异’与‘持续影响下游的差异’不是同一个概念。QK 是更值得做因果分解的边界。

7.2 L0 关键节点

节点	输入是否相同	输出 n_diff	max|Δ|	命运
input RMSNorm	是	0	0	保持一致
q_proj	是	2	0.000244140625	RoPE 后归零，瞬时
k_proj / v_proj	是	0 / 0	0	保持一致
QK matmul	是	3	4.0	传播到 softmax，持续
softmax	否或形状不同	55343	5.29e-6	概率差异扩散
L0 output	否	111	0.00390625	成为 L1 输入

8 QK 2×2：把 query 行数与 key 长度拆开

真实 S 与 B 的 QK 不只 M 不同，key 总长度 N 也不同。若直接比较两个端点，无法知道是 M、N 还是二者交互。于是使用相同的已捕获 Q/K 数值构造 2×2。比较区域始终是第 0 个 query 对共同前缀 keys 0:8209。

Cell	Query 行数 M	Key 长度 N	用途
A	1	8209	逐位复现真实 S

Cell	Query 行数 M	Key 长度 N	用途
B	1	8212	M=1 下只改变 N
C	4	8209	N=8209 下只改变 M
D	4	8212	逐位复现真实 B
E/F	4	8209/8212	将四行都重复 q1，排除其他 query 内容

比较	改变因素	raw n_diff / max	scaled n_diff / max	解释
A vs C	只改 M	3 / 64	3 / 4	完整复现真实端点差异
A vs B	M=1 下只改 N	2 / 8	2 / 1	N 有条件效应
C vs D	M=4 下只改 N	0 / 0	0 / 0	N 在 M=4 时无效
C vs E, D vs F	只改其他行内容	0 / 0	0 / 0	是形状效应，不是内容效应

raw matmul 在除以 sqrt(128) 之前已经不同，因此边界位于 CUDA BF16 matmul 本身，不在缩放。CPU 上同一组 2×2 全部逐位一致，而 CUDA 上稳定复现。这支持 kernel、tiling 或归约顺序随形状改变的解释，但没有 profiler 证据，不能点名具体 kernel，更不能称为库 bug。

9 因果干预：把相关性变成可验证的恢复链

观察实验告诉我们差异在哪里出现；因果实验则把某个边界改成 AR 的执行几何，并检查后续是否恢复。所有干预都默认关闭，只作用于 L0 指定模块，退出 context 后恢复。它们不是生产修复，而是反事实工具。

9.1 serial-causal QK

研究问题	如果 B 路径每个 query 都用 M=1 和真实 causal key prefix，当前 token 是否恢复?
为什么这样设计	B 的第 i 行单独计算 QK，M=1，K 只取 p+i+1，再把每行 scores 拼回四行张量，继续走原 mask、softmax、AV 和后续层。
关键控制	第 0 行 QK 必须与 S 逐位一致；未来列必须全部被 mask；其余层保持原 block；两次重复一致。
观测结果	QK 决策行逐位恢复；L0 输出差异 111→61；最终 token 659→323，但 logits 仍有 125439 个不同坐标。
结果如何推动下一步	QK 被确认是当前 token 翻转的因果来源之一，但 L0 仍有其他持续性边界。首个剩余边界是 o_proj。

9.2 再对齐 o_proj

研究问题	当 attention head merge 已一致时，o_proj 的 M=1/M=4 是否独立产生差异?
为什么这样设计	保留 serial QK，把四行 o_proj 改成对同一个 nn.Linear 做四次 M=1 调用再拼回。
关键控制	heads_merged 必须一致；o_proj 权重、dtype、模块实例相同；只作用于 L0。
观测结果	o_proj 决策行逐位恢复，attention residual 与 post-attention RMSNorm 也恢复；L0 输出差异 61→21。
结果如何推动下一步	attention 子块闭合，剩余差异来自 MLP。gate_proj 与 up_proj 在相同输入下独立产生差异。

9.3 闭合 MLP 的 gate/up/down

研究问题	把 MLP 的输入投影和输出投影都改为逐行 M=1，L0 能否完全恢复?
为什么这样设计	依次构造 BQOGU 与 BQOGUD：先对齐 gate/up，再加入 down。所有模块仍调用原 nn.Linear，只改变每次调用处理的 token 行数。
关键控制	post-attention RMSNorm 输入逐位一致；gate/up/down 每行输入 shape 正确；原模块、权重和 BF16 不变。
观测结果	gate/up 对齐后 L0 差异 21→5；此时 gated product 已一致，但 down_proj 仍有 12 个差异。再对齐 down_proj 后 L0 输出 5→0，与 S 逐位一致。L1 输入一致，L1 输出重新出现 222 个差异。
结果如何推动下一步	L0 的全部持续性差异完成因果闭环；同类现象在下一层重新出现，说明它是跨层系统性执行几何问题，而不是 L0 特例。

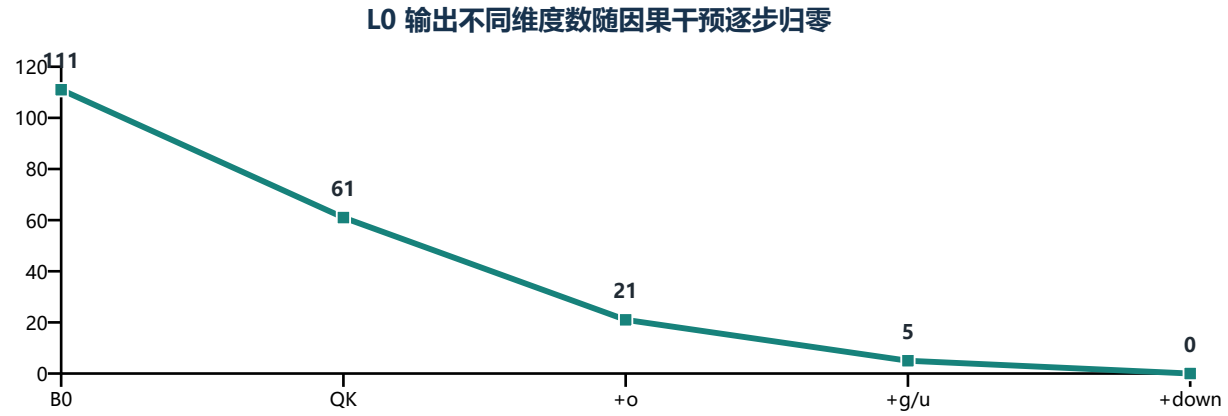


图 1 L0 输出差异随逐项因果干预归零。QK、o_proj、gate/up、down 是四组干预，但对应五个算子位置。

9.4 最终 L0 闭环表

路径	QK	o_proj	gate/up	down	L0 n_diff	最终 token
B0	block	block	block	block	111	659
BQ	M=1	block	block	block	61	323
BQO	M=1	M=1	block	block	21	323
BQOGU	M=1	M=1	M=1	block	5	323
BQOGUD	M=1	M=1	M=1	M=1	0	323

10 最终技术解释

10.1 这不是一个孤立 bug

L0 中被确认的持续性边界共有五个算子位置：QK matmul、o_proj、gate_proj、up_proj、down_proj。它们的共同特征是 BF16 GEMM 的对应行输入和权重相同，但整体 M 从 1 变为 4。逐行恢复 M=1 后，L0 完全逐位恢复；L1 随即重新分歧。这种重复性更符合执行几何导致的有限精度非不变性，而不是某个算子的公式错误。

10.2 为什么不同形状会产生不同结果

矩阵乘法的每个输出都是许多乘加项的归约。实数加法满足结合律，但浮点加法每一步都要舍入，因此 `(a+b)+c` 与 `a+(b+c)` 可能不同。GPU 库会根据 M、N、K、dtype、布局和硬件选择 kernel 与 tile。M=1 和 M=4 可能采用不同的归约树，最终落到相邻 BF16 格点。只要误差处于正常 ULP 量级，这不是数学公式改变，也不能直接定性为实现缺陷。

```
Real arithmetic: (a + b) + c == a + (b + c)
Floating point: round(round(a+b)+c) may differ from round(a+round(b+c))
SpecDec consequence: same row semantics + different GEMM geometry -> nearby BF16 result
```

10.3 为什么一个 ULP 能改变 token

位置 8209 的两个候选恰好位于 BF16 格点边界。S 路径中 token 323 与 659 都是 23.75，margin=0；B 路径把 323 推到 23.625，而 659 仍为 23.75，margin=0.125。argmax 是离散操作，因此连续数值中的一个格点变化会直接变成不同 token。随后不同 token 又写入 cache，使未来历史进一步分叉。

解释边界

‘token 恢复’也不等于‘更稳定’。干预后 margin 回到 0，argmax 只是再次按并列规则选择 323；它没有获得更大的安全余量。

11 局部定位实验排除了什么

本节排除结论仅指上述受控样本与所固定条件，不是排除所有输入上的同类错误，也不把数值行为外推为所有生产分歧的唯一原因。

候选问题	结论	证据
draft 提案或接受逻辑错误	不是当前局部根因	只加载 target 的 S/B 对照仍复现；replacement 与 round invariants 通过。
causal mask 看见未来 token	排除	未来槽位替换不改变决策行；阳性对照确认工具有效；mask -inf 数量正确。
KV cache 内容不同	在位置 8209 局部实验排除	H(8208) 的 56 个 K/V 张量逐位相同。
Embedding 或位置不同	排除	L0 输入与 embedding 决策行逐位一致。
RoPE 表来源仍不同	排除为剩余主因	CPU-FP32 inv_freq 修复后与 HF 56/56 KV 逐位一致；AR/Spec 仍分歧。
LM head 行数	排除为 token 翻转原因	固定隐藏向量换投影形状不翻转。
Final RMSNorm 行数	排除	固定输入时第 0 行逐位一致。
L27 才产生差异	排除	L27 输入已经 1357/1536 维不同。
cuDNN 融合导致	排除为唯一原因	全部关键实验使用 all-eager。
FP8 KV 量化导致	排除	关键定位使用 base_bf16 KV。
随机性或温漂	排除	同一配置、独立 cache clone 和跨 run 均逐位重复。

12 机制结论的边界与后续验证

- 没有 profiler 证据，因此尚不能点名具体 cuBLAS/cuBLASLt kernel、tile 或归约树。
- 没有证明 323 或 659 哪一个更接近高精度数学真值；本阶段没有做全模型 FP32。
- 局部位置 8209 的因果闭环不能直接外推到所有 prompt、位置、长度、模型或硬件。
- 全层逐行模式的后续实验见第13章：已知案例恢复不等于新输入泛化通过，也不构成完整 B6 输出保证。
- 逐行模式的历史性能代价已测量，见第13章；不把单案例 all-eager 结果当作当前配置的正式速度结论。
- 最终 logits 的 n_diff 不单调，不能用它替代 token-level B6 或质量指标。

13 从局部修复到候选验证：为什么没有直接解决全部分歧

前文的 111 → 61 → 21 → 5 → 0 是位置 8209 上 L0 输出不同维度数的局部干预结果，不是全模型通过率。L0 闭合后 L1 再次出现差异，说明逐层处理可能继续有效，但工程上需要评估完整生成和开销，而非继续把局部闭环当成通用结论。

13.1 全层 row-aligned 候选

研究问题	把同样的逐行规则作用于 target 全部 28 层，能否恢复已知失败案例?
为什么这样设计	只包住 target verify：28 个 QK 和 112 个 o/gate/up/down Linear 按行 M=1 执行；draft、prefill 与 AR 不启用。原权重、BF16 和原算子保持不变。
关键控制	默认关闭、模块身份注册、异常退出恢复；AR / 原 Spec / aligned 分开运行，完整 token IDs 比较，两次独立重复。
观测结果	F13 原 Spec 在 index 47 分歧，aligned 与 AR 一致；F06 也一致。单位置 token 恢复不等于 logits 逐位恢复。历史单案例性能观察：aligned decode 为原 Spec 的 1.26 倍，固定 verify 为 1.69 倍；相对 AR 吞吐为 0.176 倍。
结果如何推动下一步	有限案例上的判决 CORRECT_BUT_NO_SPEED_VALUE。该模式适合诊断，不设为生产默认；速度数字是历史 all-eager 单案例观察，非当前正式 benchmark。

13.2 四个当时未见 family 的泛化检查

研究问题	已知 F13/F06 上的恢复能否推广?
为什么这样设计	预登记 F01/F02/F08/F10，AR / production / diagnostic 各两次，共 24 次；输入重建并核 hash。禁止按结果挑样本。
关键控制	相同 BF16、base KV、all-eager、ctx8192、chunk2048、k4、max_new64；检查 hook 作用域、缓存、有限性与重复性。
观测结果	F01 diagnostic == AR；F02/F08/F10 diagnostic 不一致，首分歧为 9/52/17；production 首分歧为 14/7/52/8。所有模式重复稳定，结构检查通过。
结果如何推动下一步	ALGORITHMIC_GATE_FAIL 原样保留。row-aligned 只修改部分计算几何，不能作为完整 AR 数值轨迹的通用保证；因此转向真正单步 target oracle。

不要把局部成功升级为全局修复

这里有两个不同结论：候选在两个已知案例上恢复输出；候选在新的四例中只恢复一例。二者不矛盾，后者限制前者的泛化范围。不能因为后来采用新协议，把这一轮 FAIL 改成 PASS。

14 最后采用的方法：serial oracle 与生产路径分开

最终没有通过修改一个舍入公式来强制生产与 AR 一致，也没有用 margin 小来自动豁免。处理方法是把需要证明的事情拆开：控制逻辑是否正确；批量验证在当前有限精度下产生什么行为；这些生产轨迹的性能与质量能够支持什么声明。这个拆分经过明确批准，不是把失败隐藏在新名字下。

路径	实际计算与用途	能够证明 / 不能证明
AR	只加载 target，生成期单 token 前向；比较基线。	记录指定配置下的 AR 轨迹；不是无误差数学真值。
Serial oracle	保持 draft 提议、接受/拒绝、回滚和提交控制流程；target 判断逐 token 执行，生成期 s=1。诊断模式。	与相同输入 AR 完整一致，支持被测控制逻辑与缓存合同；不证明生产批量路径输出相同，不进入 speedup。
Production Spec	正常 draft 提议和 target 批量验证，保留块执行。	用于研究真实算法路径；必须独立记录数值分歧、硬检查、工作量与性能。

14.1 一次 serial 验证在数据上改变了什么

```
shared committed prefix
draft -> candidate[0:k]
serial target -> decisions one token at a time (s=1)
compare -> accepted prefix / first rejection
trim + repair -> committed target/draft cache
repeat until EOS or budget
```

关键不在最终把 block 的输出切成几行，而在 target 真正按照单 token 前向建立决策与历史。先用块计算再取一行，仍保留块执行的数值结果。将已有 BF16 cache 转成 FP32，也不会重新计算已经舍入过的历史。

14.2 两轮 oracle 证据不能混成一个无条件大 PASS

证据	结果	边界
历史五 family	F13/F01/F02/F08/F10：serial 与 AR 的 64 个 token 完整相同、独立重复一致。	辅助证据；旧 F13 的清理后模式读数不具备新 D6 同口径证据。
Stage 2b 新口径	F09/F03/F11/F04 × AR/prod/serial × 2 = 24 次；serial 全部与 AR 一致。production F03@59、F11@49；F09/F04 一致。	这一轮 all-eager，不是 cuDNN 证据；SERIAL_ORACLE_PASS 不等于 production exact-id PASS。

同配置重复一致说明差异可复现，并不证明差异无害。第一处分歧前可以进行同历史对照；分歧后两侧 cache 历史不同，不能直接把后续 tensor 差归为同输入算子误差。

15 B6 收尾：覆盖、验收器修正与真实 EOS

15.1 不是只有一张 token equality 表

轮次	目的	已记录结果
字面类型覆盖	10 固定用例 × AR/prod/serial × 2 = 60 次，检查自然语言、数学、代码、短/长输入与分支。	9 CASE_PASS、1 CASE_INCOMPLETE、0 FAIL；serial==AR 20/20；long production 在 idx13 分歧且重复稳定。
8K 融合复核	F09 × 3 模式 × 2 = 6 次，确认真正走长上下文 prefill 融合。	AR prefill 融合112次、双模型208次，最大块2048；不在其他阶段融合。是正确性/运行证据，不是计时稳定性结论。
CPU EOS matrix	用实际控制流与 StubModel 构造真实模型不易触发的 EOS 分支。	10 分支、29 断言；含阳性/阴性与变异体控制。支持控制逻辑，不是模型数值证据。
真实 EOS 补测	独立预登记 eos1，AR/prod/serial 各两次，共6次。	6/6 输出 EOS151645，gen_len10、stop_reason=eos、重复一致、硬检查通过；只实测全接受停止分支。

15.2 两个 harness 问题：修规则，不改数值以迁就测试

可选检查未执行：没有 mismatch 的全接受 run 不需要 repair。required=False、executed=False、ok=None 是合法的覆盖记录，不是失败。修正验收器后，必需检查仍必须执行且通过，failures 必须为空，可选 uncovered 要能逐条核对理由；不是把所有 uncovered 无条件放行。

1-token prefill：Hello 在同 tokenizer 下只有一个 token，按已有 s<2 策略合法回退 eager。旧 assertion 要求融合至少一次，因而误拒合法输入。修正策略断言以识别完整的短块回退记录，但不允许任意无融合都 PASS，也不允许非 prefill 融合。短输入通过不算融合路径覆盖，8K 另做真实复核。

15.3 EOS cache 长度为什么可以差 1

真实补测 prompt 长30，生成10个 token，最后一个是 EOS。AR 输出 EOS 后直接停止，不再把 EOS 喂回模型，因此终止 cache 为39；Spec 全接受时验证块已计算并提交 EOS，因此为40。这不是要求两侧 cache 数值一致的同历史实验，也不是缺一个 token 的 bug，而是两种终止合同。

```
AR: prompt(30) + consumed generated tokens(9) = cache 39
Spec: prompt(30) + committed generated tokens(10) = cache 40
```

验收分别检查 AR 的输入消费量与 Spec 的逐轮长度不变量，而不是强行令终止 cache 对称。旧 draft EOS 缓存缺一的真实 bug 与此合法合同差异是两件事；不能混淆。旧 eos-prone 没触发 EOS 的 CASE_INCOMPLETE 仍保留，由独立 eos1 补足证据。

16 最终如何判定与报告，而不是怎样掩盖差异

类别	判据与处置
硬失败	NaN/Inf、OOM/原生崩溃、权重或输入校验失败、必需检查缺失/失败、缓存或逐轮不变量失败、非法停止、同配置重复不一致等：保留证据并按协议停止/待查。环境拒绝不是已测样本。
生产数值差异	硬检查、有限性与长度检查通过，但 production 与 AR token IDs 不同：保留完整序列、common prefix、首分歧及两侧 token。不能自动归因近并列、不能自动宣称质量无损。
有限范围控制逻辑 PASS	serial==AR 与类型/分支/EOS 证据共同支持所测 BF16 范围；不外推所有输入、全部长度或 FP8。
性能观察与正式声明	生产时间可以记录，但输出不同时必须标为不同轨迹比较，记录接受率、长度、轮数和调用量；诊断 serial/row-aligned 时间不得冒充生产加速。

16.1 截至本次更新的状态

状态	当前结论
STAGE2B_PASS	成立，限4个新 family 的新口径 oracle 测试。
B6_BF16_PASS	2026-10-05 获明确批准，有限范围；不要求 production 永远与 AR exact-id 相同。
B6_VFP8_PASS / FULL_B6_PASS	未达成；BF16 证据不替代 V-only FP8 自己的验证。
BF16_FORMAL_READY	未达成；正式 runner、pilot 与环境/预算/记录准入仍需完成。
生产 exact-id / 生成质量	没有全局等价保证；分歧是否损伤质量尚不能由本报告回答。

直接回答：最后到底怎么处理问题

最终处理方法是：保留必要的真实实现修复；不采用无法通用保证且有成本的逐行模式作为生产默认；用 serial oracle 隔离控制逻辑；生产批量验证保留并记录数值行为；有限范围批准与正式测量资格分别管理。它解决了验收与推进方法，不等于消除了生产 token 分歧。

16.2 有哪些说法必须修正

- 旧报告的下一道门已执行：不是仍等待全层逐行候选，而是候选被评估、泛化未通过，随后完成 oracle 与 B6 收尾。
- M=1/M=4 局部因果链不等于已经点名唯一 CUDA kernel 或解释全部分歧。
- margin/ULP 可以作为诊断证据，不自动豁免；旧 exact-id FAIL/CHECK 仍保留。
- 真正的算法接受条件是匹配 target 的本轮决策，不是事后比较 AR 轨迹再拒绝。强制按 AR 修正将改变算法路径和成本，不能悄悄混进生产实验。
- 固定真实文本的 FP8 NLL/PPL 只回答 KV 表示的预测损失，不回答 AR/Spec 分歧后的生成质量。

17 从正确性收尾到正式实验

当前候选固定 BF16、target1.5B/draft0.5B、k4、chunk2048、max_new64、greedy、target RoPE CPU-FP32；cudnn_prefill 只融合 prefill，其他阶段 eager，诊断干预关闭。主实验为2K/4K/8K × AR/Spec 共6条件；V-only FP8 是独立次要消融。

正式 FF01–FF16 清单已经构建：CPU 18项检查通过，48个前缀可重建并核对长度/hash。pilot使用开发 family F09/F03/F11/F04，正式集合不按pilot接受率或速度筛选。每family×condition合计5次测量，两个测量block按3+2分配。

正式代码由用户编写。下一步是 runner 的CPU dry-run、schema与失败记录校验，然后pilot验证当前配置运行资源和计时稳定性。8K能稳定通过、需修复或受限报告均应按预登记规则处理，不能用旧 eager 波动提前否决。

17.1 复盘时按这个顺序追证据

docs/row_aligned_candidate_report_2026-10-03.md
已知案例恢复与历史性能代价
docs/correctness_protocol_validation_2026-10-03.md
四个当时未见family的row-aligned泛化失败
docs/serial_oracle_stage2_2026-10-03.md
历史五family oracle与复位字段边界
docs/stage2b_result_report_2026-10-05.md
新口径24runs，不能混称cuDNN证据
docs/b6_status_and_evidence_index_2026-10-05.md
批准、72次B6相关真实运行与CPU证据、状态边界
docs/config_freeze_2026-10-06.md
当前候选配置
docs/formal_runner_spec_2026-10-06.md
用户编写测量流程的交接规格

本报告整理仓库文档中的既有结果，没有新增GPU运行或改生产数值路径。正式采集尚未完成，未声明加速、未声明分歧无害，也未声称每一处生产差异的唯一根因已被证明。

附录 A 关键实验与提交时间线

提交	实验/结论
3f64540	仅对齐 RoPE inv_freq 来源后，自研 C1 与 HF 的 56 个 KV 张量和 logits 逐位一致。
563c53e	RoPE 对齐后真实 AR/Spec 仍在位置 8239 分歧。
f6335f5	位置 8239 的决定因素是已分叉的 cache 历史；结构逻辑检查通过。
ae24982	串行重建历史后，新首分歧出现在位置 8209。
66d3646	同历史固定输入 S/B 对照证明执行形状可以直接翻转 323/659。
fcdd5f2	LM head 不是首次来源，翻转由输入隐藏向量决定。
9c1e4f7	Final RMSNorm 行数不改变对应行，输入在此前已经不同。
60840e1	L27 输入已不同，L27 自身形状效应不足以翻转本例。
d472a4c	全层捕获确定首次分歧层为 L0。
f745357	L0 首个瞬时分歧为 q_proj；QK 在输入重新一致后独立产生差异。
aeae146	QK 2×2 证明 M=1→4 足以复现真实差异，且与 N 存在交互。
2ae7b96	serial-causal QK 恢复 token，但 L0 仍有 61 个差异，o_proj 成为持续边界。
f9cc484	再对齐 o_proj 后 attention 子块闭合，L0 剩 21 个差异，来自 MLP。
468eeea	对齐 gate/up/down 后 L0 逐位闭合，第一处分歧移动到 L1。

附录 B 关键产物索引

以下路径均相对于项目根目录 D:/RealDSH/specdec。

`docs/c1_rope_source_alignment_2026-10-01.md`
RoPE 来源对齐与 HF C1 逐位一致

`docs/ar_spec_aligned_rope_2026-10-01.md`
RoPE 对齐后的真实 AR/Spec 分歧

`docs/divasc_index47_localization_2026-10-01.md`
位置 8239 的历史效应定位

`docs/div8209_samehistory_sb_2026-10-02.md`
位置 8209 同历史 S/B 四格

`docs/div8209_chain_boundary_2026-10-02.md`
全 28 层边界与 L0 首发

`docs/div8209_l0_operator_boundary_2026-10-02.md`
L0 算子级边界

`docs/div8209_qk_2x2_2026-10-02.md`
QK 的 M/N 2×2

`docs/div8209_serial_qk_causal_2026-10-02.md`
QK 因果干预

`docs/div8209_serial_qk_oproj_2026-10-02.md`
QK + o_proj 因果干预

`docs/div8209_serial_l0_closure_2026-10-02.md`
L0 MLP 完整闭环

附录 C 复盘时应记住的五句话

- 先证明两边比较的是同一个位置、同一份历史和同一个可见前缀，再讨论数值差异。
- 最早出现差异的算子不一定是最终持续差异的来源，差异可能被后续舍入吸收。
- 观察定位边界，干预建立因果；没有恢复实验，就不要把相关性写成根因。
- 逐位一致是强控制，不是所有正确性问题的最终目标；验收应以各轮预先登记的协议为准，旧 exact-id FAIL/CHECK 不因新协议获批而改写。
- 当局部修复必须把每层 block GEMM 退化为多次 M=1 时，问题已经从调试转为工程权衡。

历史定位依据截至 commit 468eeea 的记录；第13-17章补入截至2026-10-07的候选、oracle、B6批准与正式输入准备证据。数字引用对应仓库报告，本轮未重跑GPU；旧FAIL/CHECK不改写。